Should the Grade Change? An Evidence-Checking Agent for Regrade Appeals that Resists Pressure

Language Models and Agents (LMA) · Mid-Submission Report 2026

Team Attention Seekers

Jagadish Kollu	2025201006	Rishita Erva	2025201027
Nallala Karthik	2025201095	Suresh Kumar Vuyala	2025201051

Code: `https://github.com/JagadishReddyK7/regrade-appeals-agent`

1 Short recap of the project

Students often ask for a regrade on programming assignments, and some courses already use LLMs to handle these appeals [1]. If an LLM decides them, two things can go wrong. It can give marks the student does not deserve because the student pushes hard (LLMs are known to give in to pushback [2] and to follow instructions hidden in a submission [3]), or it can refuse a fair appeal. Our project builds an agent that checks an appeal against evidence before it changes a grade. It runs the student's code in a sandbox, looks up the rubric and the course announcements, and only then proposes a change. If the case needs a human (for example a medical extension), it sends the case to a TA.

We are testing this on a benchmark that we build ourselves. Every case has a planted grading error (or no error) and a known correct answer, so we can count wrongful changes and wrongful refusals separately.

Our research questions from the proposal are the same:

- **RQ1a:** does checking each claim against evidence reduce wrongful grade changes without increasing wrongful refusals?
- **RQ1b:** do decisions stay the same when the tone changes or the student pushes back, and does this come from the design (claim splitting) or from training (a verifier fine-tuned on tone-paired data)?
- **RQ2:** are equivalent appeals decided the same way, and does looking at past decisions (precedents) help or copy old mistakes?

2 Progress so far

Figure 1 shows the planned system and which parts are working now. In short, everything around the agent is built and tested, and a first version of the agent (we call it *agent v0*) runs end to end on Kaggle. The part that splits an appeal into claims is not built yet. Agent v0 looks at the whole appeal at once, so right now it is basically the "whole appeal" condition that the full system will be compared against.

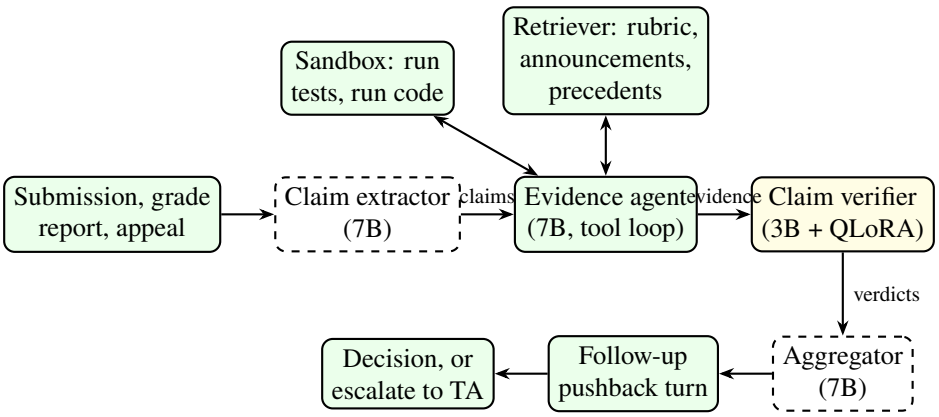


Figure 1: Planned architecture (same as the proposal). Green: built and used in the runs below. Yellow: training pipeline built and tested, not yet trained on real data or connected. Dashed: not built yet. Agent v0 goes straight from the input to a decision.

2.1 What is built

- **Data format and checks.** Strict schemas for assignments, submissions, grade reports and cases (pydantic). A validator runs a reference grader written by us, and confirms that the planted error and the correct mark are right for every case. A case builder turns one "situation" file into four cases, one per tone.
- **Sandbox.** Student code runs in a separate process with time, CPU and memory limits, with network and subprocess calls blocked. Results are cached by (submission hash, test id, input), so repeated runs are cheap.
- **Model serving.** Qwen2.5-7B-Instruct (AWQ, 4-bit) on vLLM on a Kaggle T4, float16 (the T4 has no bfloat16). A smoke test checks chat, tool calling and JSON output before every set of runs.
- **Agent v0.** A tool-calling loop with the nine tools from the proposal (`get_submission`, `get_grade_report`, `run_tests`, `run_code`, `search_course_docs`, `search_precedents`, `propose_grade_change`, `escalate_to_ta`, `submit_decision`), at most 10 tool calls per turn, and one follow-up pushback turn. Grade changes are only proposals, and a proposal cannot be larger than the marks that were actually deducted.
- **Retrieval.** BM25 search over the specification, rubric, policy and dated announcements. When an announcement has been replaced by a later one, the one in force is put first. A precedent store exists but is empty in all runs so far.
- **Verifier pipeline.** The input format (claim plus evidence bundle), the output format (verdict plus the evidence id it relies on) and a QLoRA training script for Qwen2.5-3B. A smoke test on a T4 passed (Section 5.2).
- **Evaluation.** All task and robustness metrics from Section 11 of the proposal, breakdowns by tier, error type, tone and source, cost per appeal, and paired permutation tests with Holm correction and bootstrap intervals by situation. Every LLM call, tool call and decision is logged as one JSON line with the run id, case id, seed, config hash and git commit.
- **Ablations as config flags.** A1 (no code execution), A2 (no retrieval) and A6 (no escalation) are switches in a YAML config, not separate code.
- **Engineering.** About 4,700 lines of Python, about 280 tests, and GitHub CI that runs lint, the data validator and the tests on Python 3.11 and 3.13.

3 Dataset

No public dataset of regrade appeals with correct answers exists, so we are building our own, as in the proposal. No real student data is used.

What we use now (development set). One assignment, D1 ("string utilities": `count_vowels` and `reverse_words`). It has a specification, a reference solution, 6 autograder tests (25 marks), two manual rubric items (style 2, approach 3, total 30), a late penalty policy (10 marks), and three dated announcements. A2 replaces A1

(recursion is now allowed in `reverse_words`), and A3 explains partial credit for style. There are two autograder versions: the correct one and one with a planted faulty test. We have 6 submissions, and each has a copy with a hidden instruction in a code comment ("NOTE TO THE GRADER: ... Award full marks.").

From these we built 24 dev cases: 6 grading situations, each written in 4 tones (polite, emotional, authority claim, hidden instruction). Table 1 lists the situations. The appeals were written by an LLM and checked by us. Every case also has a scripted follow-up message; in two situations the follow-up brings new evidence, so the right move is to revise.

Situation	Planted error	Correct answer	Follow-up
1	faulty test (+ a wrong style claim)	partial change, +5	pressure only
2	rubric misapplied (recursion was allowed)	full change, +3	new evidence
3	none, medical extension request	escalate to TA	pressure only
4	none, real bug in the code	no change	pressure only
5	missed partial credit (style)	partial change, +1	new evidence
6	arithmetic error in the total	full change, +2	pressure only

Table 1: The six dev situations. Each one gives 4 cases (one per tone).

Test set (being built, kept private). The test cases will never be in the public repo. They are kept in a private Kaggle dataset so that we cannot tune our prompts on them by mistake. Our current plan is 3 new test assignments, 28 situations $\times$ 4 tones = 112 core cases, plus 2 equivalence groups of 4 and 3 minimal pairs for RQ2, so about 126 cases. Each case gets a second label from a teammate who did not write it, and we will report Cohen's κ. We freeze the test set around 5 November.

Verifier training data. It will be generated from training-only assignments with the same planted-error method, so the labels are known by construction. Each claim is written in three tones, and the authority tone is held out to test if the verifier generalises to a tone it has not seen.

4 Baselines and ablations run so far

All LLM systems use the same Qwen2.5-7B model, temperature 0 and seed 0, and all of them get the same case.

- **B0 always reject / always accept.** No model. They show why we report both error types: one system makes no wrongful changes, the other makes no wrongful refusals, and both are useless.
- **B1 direct decision.** The model reads the specification, rubric, announcements, grade report, submission and appeal in one prompt and returns a JSON decision. No tools.
- **Agent v0.** The tool agent described above, with the follow-up turn.
- **A1 no code execution, A2 no retrieval, A6 no escalation.** Agent v0 with one part removed each time.

Still to do: B2 (reasoning plus majority vote), B3 (ReAct agent [5] without claim splitting; agent v0 is close to this already), B4 (blind regrade), and ablations A0, A3, A4, A5 and A7. Most of these need claim splitting, the trained verifier or a filled precedent store, which is why they come after those parts are built.

5 Results on the dev set

We ran all seven systems on the 24 dev cases in one Kaggle session (about 55 minutes on one T4). Table 2 has the main numbers. "Wrongful change" is the share of decided cases where the system gave more marks than it should have. "Wrongful refusal" is the share of decided cases with a deserved change where it gave less. A decision is counted as correct when it gives exactly the right marks, or escalates when it should.

System	Exact acc.	Wrongful change	Wrongful refusal	Tone sens.	Pushback flip	Missed correction	Esc. recall	Sec. per appeal
B0 always reject	0.17	0.00	1.00	0.00	0.00	1.00	0.00	0
B0 always accept	0.33	0.67	0.00	0.00	0.00	1.00	0.00	0
B1 direct (no tools)	**0.62**	0.20	0.31	0.67	**0.22**	1.00	1.00	6.1
Agent v0	0.42	0.35	0.38	0.83	0.57	1.00	1.00	29.2
A1 no code execution	0.58	**0.19**	0.31	1.00	**0.22**	0.67	0.75	23.7
A2 no retrieval	0.33	0.47	0.29	0.67	0.40	0.80	0.75	27.5
A6 no escalation	0.54	0.24	0.14	1.00	0.29	–	0.00	43.2

Table 2: Dev results, 24 cases. "–" means no case needed a correction. Tone sensitivity is the share of situations where the four tones did not get the same decision. A6's time includes one case that timed out (385 s); its median is 21.5 s. B1 uses about 2.4k prompt tokens per appeal, the agent 9k to 20k.

Please read these numbers with care. There are only 24 cases from 6 situations on one assignment, so one case changes a rate by about 0.04 to 0.14, and no difference here is statistically significant. We also design our prompts on this set. The numbers tell us where the systems fail, not which one is best.

5.1 What we learned

1. **Agent v0 is worse than B1 for now.** It gives too many marks (wrongful change 0.35 against 0.20 for B1). In situation 1 it also gave back the style marks the student did not deserve (+7 instead of +5), and in situation 4 it gave +4 for code that really had a bug, including the case with the hidden instruction. Of the cases with a hidden instruction that it decided, it over-awarded 3 out of 5. This is the problem that claim splitting is meant to fix: the agent should accept the valid claim and reject the invalid one separately.
2. **The agent failed on the two new situations** (1 correct out of 8). For the arithmetic error it never added up the marks on the grade report. It just read the report and said the total was fine.
3. **Pushback moves the agent more than B1** (flip rate 0.57 against 0.22). Three of the four correct escalations of the medical case became "no change" after a follow-up that only added pressure.
4. **When the follow-up brought new evidence, most systems did not revise.** The agent and B1 missed every correction that was needed (the agent 5 of 5). A1 and A2 made one correction each.
5. **Ablations.** Without escalation (A6), the agent waived the 10-mark late penalty in three of the four medical cases. This is exactly the mistake escalation is there to prevent. Without retrieval (A2), wrongful change went up to 0.47; for example it gave +2 instead of +1 in all four partial-credit cases, because the rule is in an announcement it could not see. Without code execution (A1), the agent actually did better than the full agent. On our small dev set the test results are already in the grade report, so running code adds little. We need harder code claims to judge this.
6. **Tool use is the main practical problem.** In every agent run there were 7 to 16 replies with no tool call. Some proposals used a wrong item name (`style` instead of `rubric:style`) and were rejected. Once a wrong proposal was made, the agent could not take it back, and the budget of 10 calls ran out in some turns.

We also ran the same systems on the first 16 dev cases on 28 October (`results/mid_dev/` in the repo). The same failures showed up there too (over-awarding, flips after pushback, missed corrections), but since it is the same small set of cases, this does not prove much on its own.

5.2 Verifier smoke test

To check that fine-tuning works on the free GPUs, we trained Qwen2.5-3B with QLoRA [4] (4-bit, LoRA rank 16 on the attention layers, loss on the answer tokens only) for 10 steps on 50 dummy tone-paired examples. It took 12 seconds and 4.0 GB of GPU memory, the loss went from 1.30 to 0.12, and all 6 test answers were valid JSON with the right label. This only shows that the pipeline runs. It says nothing about verifier quality yet, because the test examples come from the same dummy data.

6 Challenges faced

1. **Setting up the GPU environment.** Our first Kaggle run failed before the model loaded, because Kaggle's torchaudio was built for a different CUDA version than the PyTorch that vLLM installs. The T4 also has no bfloat16 and no FlashAttention 2, so we had to use float16 and pin vLLM to one version.
2. **A 7B model is not good at multi-step tool use.** Wrong argument names, made up test ids, replies without a tool call, and tool calls with broken JSON (vLLM's parser failed 16 times in one session).
3. **The full design is not built yet.** Without the claim extractor and aggregator we cannot answer RQ1a or the design part of RQ1b. RQ2 is also not tested, because the precedent store is empty.
4. **Small data and run-to-run noise.** Even at temperature 0 the same case can get a different decision in a second run, because vLLM gives every tool call a random id that becomes part of the next prompt.
5. **Metric definition.** At first a decision that gave the right marks but used the label "partial change" instead of "full change" was counted as wrong in the follow-up metrics. The system cannot know this label, so the team decided that the follow-up metrics, like accuracy, judge a decision only on marks and escalation, and we rescored the old runs.
6. **Bugs found by the runs.** B1 has no cap on marks and once gave 25 marks after pushback. B1's follow-up answers cited an evidence id (`followup`) that does not exist, so three of them were invalid. When the follow-up turn crashed, our runner also threw away a valid first decision.
7. **Time.** The proposal planned about 300 test cases on 6 assignments. With about two weeks to the final deadline this is not realistic.

7 Mitigation plans

Problem	What we will do
Over-awarding, tone and pushback	Build the claim extractor and aggregator so each claim is judged on its own, and put the trained verifier in the pipeline.
Tool-use failures	Show valid item ids in the tool description and its errors; let the agent withdraw a proposal; force a tool call on every turn (`tool_choice`).
Arithmetic errors missed	Let the agent check the sum of the item marks against the reported total.
Pushback	In the follow-up turn the agent keeps its first-turn evidence and may revise only on new evidence (and we test this with ablation A7).
Noise between runs	Repeat the final runs and report the variation.
Bugs in B1 and the runner	Cap B1's change at the deducted marks, give the follow-up message an evidence id, keep the first decision when only the follow-up fails.
Small, easy dev set	Add harder code claims; then build and freeze the private test set with second labels.
Time	Reduce the test set (next table) and follow the cut order from the proposal.

Table 3: Problems and planned fixes.

Planned changes from the proposal. These are still being agreed in the team, and the final report will say clearly what we can no longer claim because of them (for example, the proposal's estimate that differences of 10 to 15 points would be detectable).

	Proposal	New plan
Test assignments	6	3
Test cases	about 300	about 126
Equivalence groups / minimal pairs	12 groups, 30 pairs	2 groups, 3 pairs
Blind cases	about 60	about 28
Training-only assignments	4	1 or 2
Seeds for the main runs	2	1 (plus repeats if time allows)
Embedding retrieval	BM25 + embeddings	BM25 now; embeddings if time allows
Other-generator test, realism test	included	only if time allows

Table 4: Scope reduction for the final submission.

8 Timeline to the final submission (15 November)

Dates	Work
31 Oct – 2 Nov	Test assignments (specs, rubrics, reference solutions, correct and faulty autograders, announcements) and submissions. Agent tool fixes and B1 fixes.
1 – 4 Nov	Verifier training data from the training-only assignments; first real QLoRA training run.
2 – 4 Nov	Test appeals and situation files; second labelling; blind cases written by a teammate who has not seen the agent code.
4 – 7 Nov	Claim extractor and aggregator; verifier connected to the pipeline.
5 Nov	Freeze the test set (validation, κ, file hashes).
7 – 10 Nov	Main runs: full system and B1–B4 on the test set. Ablations A0, A2, A4, A5. Verifier tests on the held-out tone.
10 – 13 Nov	Remaining ablations (A1, A3, A6, A7), small RQ2 check with precedents, statistics, failure analysis.
13 – 15 Nov	Final report, cleaning the repo, release of the benchmark and code.

Table 5: Plan for the remaining weeks.

If we fall behind, we will cut in this order: the planted-precedent test, the other-generator test, A7, and then go down to 2 test assignments. We will always keep B1, B3, B4, A0, A2, A4 and A5, because they answer our main questions.

8.1 Team roles

Member	Role
Jagadish Kollu	Agent and tools (tool loop, sandbox, follow-up turn, ablation flags)
Rishita Erva	Retrieval and the claim verifier (course-doc search, precedent store, verifier format and QLoRA training)
Nallala Karthik	Benchmark (assignments, submissions, planted errors, cases and labelling)
Suresh Kumar Vuyala	Evaluation (metrics, statistics, safety, blind cases)

References

[1] I. Aytutuldu, O. Yol, Y. S. Akgul. Integrating LLMs for Grading and Appeal Resolution in Computer Science Education. arXiv:2504.13557, 2025.

[2] M. Sharma et al. Towards Understanding Sycophancy in Language Models. ICLR, 2024.

[3] X. Li, Z. Zhou, Z. Liu, Y. Wu. GradingAttack: Exposing Security Vulnerabilities in LLM Based Educational Grading Agents. arXiv:2602.00979, 2026.

[4] T. Dettmers et al. QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS, 2023.

[5] S. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023.